Maets – Annexe technique

Back-end du POC de la librairie de jeux vidéo Maets

Ce document explique comment le back-end est construit et pourquoi on a fait ces choix. L'installation pas à pas et la liste des commandes sont dans le `README.md`, et la description des routes dans Swagger (`/api/docs`).

Les parties marquées **Pas encore fait** ne sont pas encore réalisées au moment où j'écris ce document.

Sommaire

1. Vue d'ensemble
2. Environnement de développement
3. Conception du back-end : POO, MVC, clarté
4. Serveur : ressources et sécurité
5. Base relationnelle PostgreSQL
6. Base NoSQL MongoDB
7. Choix entre SQL et NoSQL
8. Sauvegarde et récupération
9. Conception de l'API
10. Sécurité de l'API
11. Tests
12. Documentation

---

1. Vue d'ensemble

Avec Maets, un joueur se connecte, consulte sa librairie de jeux, peut en retirer un et régler chacun de ses jeux. Un admin gère le catalogue et ajoute des jeux dans la librairie des joueurs.

Le projet tient en trois morceaux :

- **l'API** : Node.js 22.12 ou plus et Express 5, en HTTPS, avec des routes REST et une authentification par JWT ;
- **PostgreSQL 17** (avec Sequelize) pour les utilisateurs, les rôles, les éditeurs, les jeux et les librairies ;
- **MongoDB** (avec Mongoose) pour la configuration de chaque jeu pour chaque joueur.

Le trajet d'une requête :

```
Client ──HTTPS──► route ──► middleware (JWT, rôle) ──► contrôleur ──► service ──► repository ──► PostgreSQL / MongoDB
                                                                                          │
                                                                                DTO ◄─────┘  (mise en forme de la réponse)
```

2. Environnement de développement

2.1 Installation

Pour travailler sur le projet, il faut Visual Studio Code, Node.js 22.12 ou plus et Git. Pour les bases de données, Docker est obligatoire : on installe Docker Desktop, puis `docker compose up -d --wait` démarre PostgreSQL 17 et MongoDB 8 déjà configurés (la base et l'utilisateur sont créés à partir du `.env`). J'ai choisi une seule façon d'installer pour que ce soit plus simple : tout le monde a les mêmes versions des bases, et il n'y a rien à créer à la main.

Le `docker-compose.yml` décrit les deux conteneurs : des images dont la version majeure est fixée (`postgres:17-alpine`, `mongo:8`), des volumes nommés pour garder les données, un `healthcheck` pour savoir quand les bases sont prêtes, et les ports 5433 et 27018 pour ne pas gêner des bases déjà installées sur le poste. Je n'ai mis que les bases dans Docker : l'API reste lancée avec Node, pour garder le débogage dans VS Code et le redémarrage automatique.

Une fois le `.env` rempli et les bases lancées, quatre commandes suffisent pour avoir une API prête : `npm install`, `npm run db:migrate`, `npm run db:seed` et `npm run cert` (le certificat HTTPS).

Pour vérifier que le README suffit vraiment, j'ai refait toute l'installation sur une copie propre du projet, comme si je venais de le récupérer (sans `.env`, `certs/`, `node_modules/` ni sauvegardes), avec des bases Docker neuves et le PATH par défaut de Windows. En suivant le README à la lettre, tout a marché : installation, migrations, données de test, certificat, démarrage, Swagger, les 12 routes, sauvegarde, restauration et test de charge. Ce test m'a fait corriger un vrai problème : `npm run cert` ne trouvait pas OpenSSL, qui n'est pas dans le PATH de Windows.

Pour le test de charge seulement, il faut aussi Java et Apache JMeter 5.6.3. JMeter n'a pas besoin d'être installé : on dézippe l'archive et on indique son dossier dans le `.env` (`JMETER_HOME`).

VS Code est configuré par les fichiers du dossier `.vscode/`, livrés avec le projet :

- **Débogage** (`launch.json`) : deux configurations, « Déboguer l'API » (lance `src/server.js` avec l'inspecteur de Node) et « Déboguer les seeders ». On lance avec `F5`, et on a les points d'arrêt, l'inspection des variables et le pas à pas.
- **Autocomplétion** : celle de VS Code pour JavaScript, réglée pour les modules ES. Les imports sont générés avec l'extension `.js` (obligatoire avec `"type": "module"`) et mis à jour tout seuls quand on déplace un fichier. Les extensions conseillées ajoutent la complétion des chemins de fichiers et des noms de paquets npm.

2.2 Commandes du terminal

Chaque commande est un script npm, expliqué dans le README. On sépare bien le développement de la production :

Usage	Développement	Production
Installer les dépendances	`npm install`	`npm ci` (les versions exactes du `package-lock.json`)
Lancer l'API	`npm run dev` (redémarre à chaque modification)	`npm start`
Environnement	rien à faire (`development` par défaut)	`NODE_ENV=production` : Faker n'est pas installé
Données	`npm run db:seed` (données de démo)	`npm run db:seed:prod` (rôles et compte admin seulement)
Test de charge	`npm run load-test` (JMeter en ligne de commande, sans interface)	

La version minimale de Node est indiquée dans le `package.json` (`"engines": { "node": ">=22.12.0" }`), parce que les migrations sont écrites en modules ES et qu'il faut Node 22.12 pour les lire.

2.3 Personnalisation de l'IDE

En tant que lead dev, j'ai fixé la configuration de l'éditeur pour toute l'équipe avec le dossier `.vscode/`. Elle s'applique dès qu'on ouvre le dossier du projet, sans toucher aux réglages perso de chacun sur ses autres projets.

Fichier	Ce qui est réglé	Pourquoi
`settings.json`, apparence	Thème sombre (`Default Dark Modern`), icônes Material, repère vertical à 120 caractères, couleurs des paires de parenthèses	Tout le monde a le même environnement
`settings.json`, style	2 espaces d'indentation (sans détection automatique), formatage à l'enregistrement, suppression des espaces en fin de ligne, saut de ligne à la fin	Le code reste homogène, peu importe qui l'écrit, et on évite les différences inutiles dans Git
`settings.json`, formateur	Le formateur intégré de VS Code pour JavaScript et JSON, et Prettier déconseillé (`unwantedRecommendations`)	Un Prettier sans config remplacerait les guillemets simples du projet par des doubles dans chaque fichier enregistré
`extensions.json`	9 extensions proposées à l'ouverture : français, icônes, couleurs des `.env`, complétion des chemins et des paquets npm, SQLTools avec le pilote PostgreSQL, MongoDB, REST Client	Chacune sert au projet : on peut explorer les deux bases et tester l'API sans quitter l'éditeur
`maets.code-snippets`	`mctrl`, `mservice`, `mrepo`, `mroute`, `merr` : le squelette d'un contrôleur, d'un service, d'un repository, d'une route protégée ou d'une erreur	On respecte l'architecture en couches et on gagne du temps à chaque nouvelle fonctionnalité

Pour les raccourcis clavier : VS Code ne permet pas d'imposer des raccourcis par projet (le fichier `keybindings.json` est personnel à chaque développeur). L'équipe a donc les raccourcis de code ci-dessus, et le README rappelle les raccourcis utiles du projet (`F5` pour déboguer, `Ctrl+Maj+D` pour l'onglet de débogage).

Le mode d'emploi est dans le README, partie 2.

---

3. Conception du back-end

3.1 Programmation orientée objet

Les erreurs de l'API sont des classes :

```
Error                    (classe de base de JavaScript)
└── AppError             ajoute statusCode, code, details
    ├── BadRequestError   400
    ├── UnauthorizedError 401
    ├── ForbiddenError    403
    ├── NotFoundError     404
    └── ConflictError     409
```

- **Héritage** : chaque erreur hérite d'`AppError`, qui hérite d'`Error` (on garde le message et la pile d'appels).
- **Polymorphisme** : le gestionnaire d'erreurs traite toutes ces classes avec un seul test, `err instanceof AppError`, sans avoir à connaître chaque sous-classe.
- **Encapsulation** : dans chaque service, les fonctions internes (`validateGameInput`, `findGameOrFail`, `checkUser`…) ne sont pas exportées. Les contrôleurs ne voient que les opérations dont ils ont besoin.

Je n'ai pas utilisé l'héritage ailleurs, comme le conseille le cours : les routes REST ont peu de choses en commun, et l'héritage rendrait le code plus dur à lire. Les erreurs sont le cas où il sert vraiment.

3.2 Architecture MVC

Couche	Dossier	Rôle
Modèle	`models/`	La description des tables et de la collection
Vue	`dto/`	Le JSON renvoyé au client (par exemple, le hash du mot de passe n'est jamais renvoyé)
Contrôleur	`routes/` et `controllers/`	Reçoit la requête, appelle le service, renvoie la réponse
Service (ajouté)	`services/`	La logique : vérifier les données reçues, lever les erreurs (404, 409…), préparer la réponse avec le DTO
Repository (ajouté)	`repositories/`	L'accès aux bases, et rien d'autre : toutes les requêtes Sequelize et Mongoose sont là

J'ai ajouté ces deux couches pour alléger les contrôleurs et garder un code propre. Un contrôleur ne fait plus que 2 ou 3 lignes, toutes les vérifications d'un jeu sont au même endroit (`game.service.js`), et si on changeait un jour d'ORM ou de base, l'essentiel des changements serait dans les repositories.

3.3 Clarté du code

- Un rôle par fichier, et des noms qui disent ce qu'il y a dedans (`game.controller.js`, `game.service.js`, `game.repository.js`, `game.model.js`, `game.route.js`).
- Le même format d'erreur pour toute l'API : `{ "error": { "code", "message", "details" } }`.

---

4. Serveur : ressources et sécurité

4.1 Installation du serveur et ressources nécessaires

Installation sur un serveur Linux (production) : on installe Node.js (version 22.12 ou plus récente), puis on lance PostgreSQL 17 et MongoDB avec le même `docker-compose.yml` (Docker Engine). On copie le projet, on crée le `.env` avec des secrets propres au serveur et on met le certificat dans `certs/`. Ensuite, avec `NODE_ENV=production` : `npm ci`, `npm run db:migrate` et `npm run db:seed:prod` (rôles et compte admin). Enfin, on lance `npm start` avec un gestionnaire de services (systemd) pour qu'il redémarre tout seul.

Ressources par composant, avec les hypothèses de la partie 7 (100 000 inscrits, 200 requêtes par seconde au pic) :

Composant	Processeur	Mémoire	Stockage
API Node.js (2 instances)	1 vCPU chacune	256 à 512 Mo chacune	Presque rien (le code et les logs)
PostgreSQL	1 vCPU	1 Go (toute la base tient en mémoire, environ 350 Mo)	Environ 1 Go avec les index
MongoDB	1 vCPU	4 Go (cache WiredTiger pour environ 3,6 Go de configurations)	Environ 4 Go
Total conseillé	4 vCPU	8 Go	40 Go SSD (données, logs, 7 jours de sauvegardes)

Pour comparer, la base de démo actuelle fait moins de 10 Mo.

Paramètres réglés dans le code :

Paramètre	Valeur	Ressource	Effet
Compression gzip des réponses	au-delà de 1 Ko	Réseau et CPU	La liste des jeux passe de 5,3 Ko à 1,6 Ko. Les petites réponses ne sont pas compressées, ça ne ferait que coûter du CPU
Taille maximale d'une requête	100 Ko	Mémoire	Une configuration fait quelques Ko. Un corps plus gros est refusé (413) au lieu de remplir la mémoire
Pool de connexions PostgreSQL	10 max, 0 min, fermeture après 10 s sans activité	Mémoire (côté PostgreSQL)	Les connexions sont réutilisées. Chaque connexion coûte de la mémoire à PostgreSQL, qui en accepte 100 par défaut : avec 10, il reste de la place pour les outils (pgAdmin, sauvegardes) et d'autres instances
Pool de connexions MongoDB	10 max	Mémoire	Même idée
Délais HTTP	requête 30 s, en-têtes 10 s, connexion inactive 5 s	Mémoire	Les connexions lentes ou abandonnées ne restent pas ouvertes (ça protège aussi contre l'attaque « Slowloris »)
Arrêt propre	sur `Ctrl+C` (SIGINT) et sur l'arrêt envoyé par le serveur (SIGTERM, utilisé par systemd et Docker)	Fiabilité	Les requêtes en cours se terminent, puis on ferme les connexions aux bases

Si la charge augmente : on peut lancer plusieurs instances de l'API derrière un répartiteur de charge. Comme le JWT est stateless, n'importe quelle instance peut traiter n'importe quelle requête. Il faut juste garder le nombre d'instances fois 10 connexions sous la limite de PostgreSQL.

4.2 Sécurité du serveur : HTTPS

HTTPS (SSL/TLS) : c'est en place, l'API n'accepte que des connexions chiffrées.

Élément	Comment c'est fait
Création du certificat	`npm run cert` (`scripts/cert.js`), qui lance : `openssl req -x509 -newkey rsa:2048 -nodes -days 365 -keyout certs/key.pem -out certs/cert.pem -subj /CN=localhost -addext subjectAltName=DNS:localhost,IP:127.0.0.1`. Git installe OpenSSL sans l'ajouter au PATH de Windows, donc le script va aussi le chercher dans le dossier de Git
Caractéristiques	Clé RSA 2048 bits, certificat X.509 auto-signé valable 1 an, pour `localhost` et `127.0.0.1`
Serveur	`src/server.js` lit `certs/key.pem` et `certs/cert.pem` et démarre avec le module `https` de Node : https://localhost:3000
HTTP simple	Refusé (aucun port HTTP n'est ouvert)
Clé privée	Dans `certs/`, exclu de Git. Elle n'est jamais partagée, chaque poste crée la sienne

Ce que j'ai vérifié : une requête HTTPS avec le certificat répond 200, une requête en HTTP n'a aucune réponse, un client qui ne fait pas confiance au certificat refuse la connexion (c'est normal avec un certificat

auto-signé), et toutes les routes marchent en HTTPS.

Sans HTTPS, le token JWT et les mots de passe passeraient en clair sur le réseau.

---

5. Base relationnelle PostgreSQL

5.1 Schéma et cardinalités

```
users ──< user_roles >── roles                    users et roles : plusieurs à plusieurs
users ──< user_games >── games >── publishers     users et games : plusieurs à plusieurs (la
librairie)
                                                  games vers publishers : plusieurs à un
```

Table	Colonnes principales	Contraintes
`users`	id, email, password_hash	email unique
`roles`	id, name	name unique (`user`, `admin`)
`user_roles`	user_id, role_id	clé primaire composée, suppression en cascade
`publishers`	id, name	name unique
`games`	id, title, description, release_date, publisher_id	title unique, publisher_id obligatoire, `ON DELETE RESTRICT`
`user_games`	user_id, game_id, config_id, added_at	clé primaire composée, `ON DELETE CASCADE`, config_id unique

- Le cours propose une table `users` avec un `username`. Ici, c'est l'**email** qui sert d'identifiant : il est unique et il faut de toute façon le demander, donc un nom d'utilisateur en plus ferait doublon.
- Un utilisateur peut avoir plusieurs rôles (l'admin a `user` et `admin`).
- La clé primaire composée de `user_games` empêche d'avoir deux fois le même jeu.
- Avec `ON DELETE CASCADE`, un jeu ou un utilisateur supprimé disparaît des librairies : il ne reste pas de lignes orphelines.
- Avec `ON DELETE RESTRICT`, on ne peut pas supprimer un éditeur qui a encore des jeux.

Le schéma est créé par des **migrations** versionnées (`database/migrations/`), qu'on peut rejouer à l'identique sur n'importe quel poste.

5.2 Normalisation

- Le nom d'un éditeur n'est stocké qu'une fois (table `publishers`), au lieu d'être répété dans chaque jeu.
- Les rôles sont dans une table à part, et le lien entre un utilisateur et ses rôles dans une table de liaison.
- Il n'y a aucune donnée calculée ou en double : le schéma est en troisième forme normale.
- C'est la base elle-même qui garantit l'intégrité (clés étrangères, `UNIQUE`, `NOT NULL`), même si le code a un bug. J'ai vérifié ces contraintes directement en SQL : insérer un doublon, mettre un éditeur qui n'existe pas, supprimer un éditeur utilisé, tout est refusé.

5.3 Performance et sécurité

Performance

- Il y a des index sur les clés étrangères, que PostgreSQL ne crée pas tout seul : `games(publisher_id)` et `user_games(game_id)`. La clé primaire `(user_id, game_id)` sert pour « la librairie d'un utilisateur ».
- Les jeux sont chargés avec leur éditeur en une seule requête (jointure), et pas avec une requête par jeu.

Test de charge (JMeter)

L'outil : Apache JMeter 5.6.3, qui est cité dans le programme du module. Le plan de test est dans le projet (`load-test/maets.jmx`) et se lance en ligne de commande avec `npm run load-test`. Le test se rejoue donc à l'identique sur n'importe quel poste, et JMeter produit un rapport HTML par palier (`load-test/reports/<joueurs>/index.html`).

Le scénario : il imite ce que fait un vrai joueur. Chaque joueur virtuel se connecte **une fois** avec un compte de démo (`POST /api/auth/login`) et garde son token, puis enchaîne en boucle, sans pause :

Requête	Ce qu'elle fait travailler
`GET /api/games`	PostgreSQL : les 30 jeux avec leur éditeur (jointure)
`GET /api/games/:id` (un jeu au hasard)	PostgreSQL : recherche par la clé primaire
`GET /api/users/:userId/library` (avec le token)	La vérification du JWT et des droits, puis PostgreSQL : la librairie avec les jeux

Les paliers : **10, 50 puis 100 joueurs en même temps**, 30 secondes chacun, et les joueurs arrivent petit à petit sur les 10 premières secondes. Comme ils n'attendent pas entre deux requêtes, 100 joueurs virtuels font bien plus de travail que 100 vrais joueurs (un vrai joueur fait une requête toutes les quelques secondes).

Les conditions : l'API (`npm start`), les deux bases (Docker) et JMeter tournaient sur le **même ordinateur** (processeur AMD Ryzen 7 5800HS, 8 cœurs et 16 threads, 16 Go de mémoire), en HTTPS. JMeter prenait donc une partie du processeur pendant le test : les chiffres sont plutôt pessimistes.

Les résultats (test du 28/10/2026) :

Joueurs	Requête	Requêtes	Req/s	Moyenne	p95	Erreurs
10	`GET /api/games`	6 777	231	11 ms	16 ms	0 %
10	`GET /api/games/:id`	6 773	231	10 ms	15 ms	0 %
10	`GET /api/users/:userId/library`	6 772	231	16 ms	22 ms	0 %
10	`POST /api/auth/login`	10		163 ms	568 ms	0 %
10	**Total**	**20 332**	**680**	**12 ms**	**20 ms**	**0 %**
50	`GET /api/games`	5 881	202	57 ms	117 ms	0 %
50	`GET /api/games/:id`	5 864	203	55 ms	111 ms	0 %
50	`GET /api/users/:userId/library`	5 855	204	100 ms	173 ms	0 %
50	`POST /api/auth/login`	50		208 ms	714 ms	0 %
50	**Total**	**17 650**	**590**	**71 ms**	**142 ms**	**0 %**
100	`GET /api/games`	4 643	161	138 ms	299 ms	0 %
100	`GET /api/games/:id`	4 612	161	136 ms	241 ms	0 %
100	`GET /api/users/:userId/library`	4 586	165	250 ms	392 ms	0 %
100	`POST /api/auth/login`	100		943 ms	1 160 ms	0 %
100	**Total**	**13 941**	**466**	**180 ms**	**421 ms**	**0 %**

Le *p95*, c'est le temps sous lequel répondent 95 % des requêtes. En gros, c'est ce que vivent les utilisateurs les moins bien servis.

Ce que j'en retiens

- **Aucune erreur sur presque 52 000 requêtes** : quand il y a du monde, les requêtes attendent leur tour au lieu d'échouer.
- **Les temps restent corrects** : à 100 joueurs sans pause, 95 % des requêtes répondent en moins de 0,45 s.
- **Le débit plafonne vers 680 requêtes par seconde dès 10 joueurs** : au-delà, les joueurs font la queue (Node exécute le JavaScript sur un seul cœur, et JMeter tournait sur le même PC).
- **La connexion est lente exprès** : bcrypt met environ 75 ms par vérification pour freiner les attaques par force brute, et on ne se connecte qu'une fois toutes les 2 heures.

Sécurité

- **Des données de test anonymes** : elles sont générées par Faker, les emails sont en `@example.com` (un domaine réservé aux exemples), il n'y a aucune donnée réelle, et la graine est fixe pour avoir les mêmes données sur chaque poste.
- **Injection SQL** : toutes les requêtes passent par Sequelize, qui envoie les valeurs à part de la requête (requêtes paramétrées). Aucune valeur venant du client n'est collée dans du texte SQL.
- **Mots de passe** hachés avec bcrypt (coût 10, avec un sel aléatoire).
- **Le moins de droits possible** : l'application se connecte avec l'utilisateur `maets`, jamais avec le compte `postgres`. L'image PostgreSQL fait de `maets` l'administrateur du conteneur, mais ce conteneur ne contient que la base du projet.

6. Base NoSQL MongoDB

6.1 Pourquoi MongoDB pour les configurations

La configuration d'un jeu (graphismes, contrôles, son…) n'a **pas la même forme d'un jeu à l'autre** :


```
{ "userId": 2, "gameId": 2, "settings": { "graphics": { "quality": "high", "fov": 72 }, "mouse": {
"sensitivity": 3 } } }
{ "userId": 3, "gameId": 4, "settings": { "controls": { "steeringSensitivity": 10 }, "assists": {
"abs": false } } }
```


(Exemples simplifiés : un FPS et un jeu de course, avec une partie de leurs réglages. Les données de démo en génèrent quatre formes différentes : course, RPG, FPS et stratégie.)

Ces réglages sont dans la collection MongoDB `game_configs`, et le lien avec PostgreSQL marche dans les deux sens :

- `user_games.config_id` (dans PostgreSQL) contient l'`_id` du document MongoDB ;
- le document contient `userId` et `gameId` pour faire le chemin inverse.

C'est le code qui garde les deux bases cohérentes : quand on retire un jeu d'une librairie ou qu'on supprime un jeu, les configurations qui vont avec sont supprimées aussi. J'ai vérifié sur toutes les données que chaque `config_id` mène au bon document et qu'aucun document n'est orphelin.

6.2 Optimisation : index, réplication et partitionnement

Pourquoi c'est plus simple à maintenir en NoSQL : en SQL, il faudrait soit une colonne par réglage possible de chaque jeu (des centaines de colonnes, presque toujours vides), soit une table par jeu, avec une migration à chaque nouveau jeu. Dans MongoDB, chaque document a sa propre forme : un nouveau jeu avec des réglages jamais vus ne demande aucun changement de schéma.

Index : un index unique sur `(userId, gameId)`. Il garantit qu'il n'y a qu'une configuration par joueur et par jeu, et il rend la recherche quasi instantanée, quel que soit le nombre de documents.

Réplication (pour la disponibilité) : en production, MongoDB tourne en *replica set* de 3 nœuds (1 primaire et 2 secondaires). Si le primaire tombe, un secondaire est élu tout seul pour le remplacer. On peut aussi répartir les lectures sur les secondaires.

Partitionnement (*sharding*, pour le volume) : si les configurations deviennent très nombreuses, on répartit la collection sur plusieurs serveurs avec la clé de partitionnement `userId` (hachée). Toutes les configurations d'un joueur sont alors sur le même serveur, donc une requête n'interroge qu'un serveur, et la charge est bien répartie entre eux.

7. Choix entre SQL et NoSQL

Pour choisir, je me suis basé sur trois critères : la **performance**, le **coût** et l'**évolutivité**. Pour chacun, je pars d'hypothèses chiffrées (ci-dessous), que je compare ensuite à ce que chaque base sait bien faire.

7.1 Les hypothèses de départ (volume et trafic)

Hypothèse	Valeur retenue	Pourquoi
Inscrits au bout de 3 ans	100 000	Une plateforme qui démarre, sur un public de niche
Jeux au catalogue	5 000	Le catalogue d'une boutique indépendante
Jeux par joueur	20 en moyenne	Ça donne **2 millions de lignes** dans `user_games`
Jeux avec une configuration	60 %	Ça donne **1,2 million de documents** dans `game_configs`
Taille d'une configuration	1 à 5 Ko (3 Ko en moyenne), forme variable	Réglages graphiques, contrôles, son, propres à chaque jeu
Joueurs connectés en même temps au pic	2 % des inscrits, soit 2 000	Un soir ou un week-end
Requêtes par joueur connecté	1 toutes les 10 secondes	On navigue dans sa librairie, on ouvre un jeu
Trafic au pic	2 000 divisé par 10, soit **200 requêtes par seconde**	Dont moins d'une connexion par seconde (le token dure 2 h)
Lectures et écritures	environ 90 % de lectures, 10 % d'écritures	On consulte bien plus qu'on ne modifie

Le volume de données que ça donne :

Données	Calcul	Volume
`user_games` (PostgreSQL)	2 millions de lignes d'environ 75 octets, plus 3 index	environ 300 Mo
`users`, `user_roles`, `games`, `publishers`	100 000 comptes, 5 000 jeux	environ 50 Mo
`game_configs` (MongoDB)	1,2 million de documents de 3 Ko	environ 3,6 Go (moins sur le disque, car MongoDB compresse les données)

7.2 Performance

	PostgreSQL (librairie, comptes, catalogue)	MongoDB (configurations)
Accès le plus fréquent	« la librairie d'un joueur » : lecture par la clé primaire `(user_id, game_id)` et jointure avec `games`	« la configuration d'un jeu pour un joueur » : un document lu en entier par son `_id`
Ce qui le rend rapide	L'index B-tree : 3 ou 4 lectures de page suffisent pour trouver une ligne parmi 2 millions, quel que soit le volume	Le document est stocké d'un bloc : une seule lecture, et pas de jointure pour reconstituer les réglages
Mémoire	Environ 350 Mo : toute la base tient en mémoire, les lectures ne vont presque jamais sur le disque	Environ 3,6 Go : ça tient dans le cache de MongoDB (WiredTiger) sur un serveur de 8 Go
Temps attendu en base par requête	moins de 5 ms	moins de 5 ms

Comparaison avec le test de charge : le test (partie 5.3) a tenu entre 466 et 680 requêtes par seconde, sans erreur, avec **une seule** instance de l'API, sur un ordinateur portable qui faisait aussi tourner JMeter. Le pic prévu de 200 requêtes par seconde passe donc avec une marge d'au moins 2 fois. Avec deux instances en production (pour avoir un secours), la marge passe à environ 4 fois. Et ce ne sont pas les bases qui limitent : 200 requêtes par seconde, ça fait au plus 400 requêtes SQL simples et indexées par seconde, alors que PostgreSQL en traite plusieurs milliers par seconde sur un petit serveur. La limite vient de l'API elle-même (le processus Node), pas des bases.

7.3 Coût

Ce sont des ordres de grandeur par mois, hors taxes, d'après les tarifs publics des hébergeurs en 2026. Il faudra les revérifier au moment de choisir.

Poste	Option 1 : au lancement (un seul serveur)	Option 2 : en production, avec des secours
API	1 serveur virtuel (VPS) de 4 vCPU et 8 Go, qui héberge aussi les deux bases : 15 à 30 €	2 VPS derrière un répartiteur de charge : 30 à 50 €
PostgreSQL	compris dans le VPS	Une base managée avec un serveur de secours : 50 à 100 €
MongoDB	compris dans le VPS	Un cluster managé (*replica set* de 3 nœuds, la première offre dédiée) : environ 55 €
Stockage des sauvegardes	environ 1,5 Go par sauvegarde compressée, fois 7 jours, soit une dizaine de Go : 1 à 5 €	1 à 5 €
Certificat HTTPS	Let's Encrypt : 0 €	0 €
Licences	0 € : PostgreSQL est sous licence libre, et MongoDB Community est gratuit tant qu'on ne revend pas MongoDB comme service (licence SSPL)	0 €
Total	**environ 20 à 35 € par mois**	**environ 140 à 210 € par mois**

Le coût humain : avoir deux bases, c'est deux technos à connaître, deux sauvegardes et deux choses à surveiller. Dans ce projet, le surcoût reste petit : une seule commande (`npm run backup`) sauvegarde les deux bases, et Docker Compose les installe en une commande.

7.4 Évolutivité

Hypothèse de croissance : **10 fois plus**, soit 1 million d'utilisateurs, 20 millions de lignes dans `user_games`, 12 millions de configurations et 2 000 requêtes par seconde au pic.

Élément	Ce qui se passe avec 10 fois plus	Ce qu'on fait	Ce que la base permet
PostgreSQL	Environ 3,5 Go : ça tient encore sur un seul serveur	Un serveur plus puissant, puis une réplique en lecture pour répartir les lectures	Il n'y a pas de partitionnement natif entre serveurs (il faudrait une extension comme Citus), mais on n'en a pas besoin en dessous de plusieurs centaines de millions de lignes
MongoDB	Environ 36 Go : ça ne tient plus dans la mémoire d'un serveur de 8 Go	Du partitionnement (*sharding*) sur 2 ou 3 serveurs avec la clé `userId` hachée, sans changer le code de l'API (partie 6.2)	Partitionnement et réplication natifs
API	2 000 requêtes par seconde	5 à 10 instances derrière un répartiteur de charge	Le JWT est stateless, donc n'importe quelle instance traite n'importe quelle requête
Connexions PostgreSQL	10 connexions par instance, et PostgreSQL en accepte 100	Jusqu'à 9 instances sans rien changer, et au-delà un répartiteur de connexions (PgBouncer)	
Nouveaux jeux avec des réglages jamais vus	Ils arrivent en continu	Aucune migration : chaque document MongoDB a sa propre forme	Schéma souple dans MongoDB, schéma strict dans PostgreSQL pour les données qui doivent rester cohérentes

7.5 Le bilan par type de donnée

Critère	Librairie, comptes, catalogue	Configurations
Structure	Fixe, la même pour toutes les lignes, avec beaucoup de liens	Différente d'un jeu à l'autre
Intégrité	Critique (qui possède quoi) : clés étrangères, transactions	Moins critique : une configuration perdue, ça se refait
Performance	Environ 350 Mo, tout en mémoire, jointures indexées en quelques millisecondes (partie 7.2)	Un document lu en entier par sa clé, sans jointure (partie 7.2)
Coût	Licence gratuite, et elle tient sur le même serveur au lancement (partie 7.3)	Licence gratuite, environ 55 € par mois en managé avec des secours (partie 7.3)
Évolutivité	Avec 10 fois plus, toujours un seul serveur (partie 7.4)	Avec 10 fois plus, il faut partitionner, et MongoDB sait le faire (partie 7.4)
Choix	**PostgreSQL**	**MongoDB**

---

8. Sauvegarde et récupération

8.1 Le plan de sauvegarde

C'est en place : deux scripts (`scripts/backup.js` et `scripts/restore.js`) et une tâche planifiée tous les jours.

Élément	Dans le POC	En production
Quoi	Toute la base PostgreSQL (`pg_dump`, format compressé `custom`) et toute la base MongoDB (`mongodump`, archive gzip), lancés dans les conteneurs Docker	Pareil
Quand	`npm run backup` tous les jours à 3 h (quand il n'y a presque personne), avec le Planificateur de tâches Windows	Avec `cron` : `0 3 * * * cd /chemin/vers/backend && npm run backup`
Cohérence	Les deux bases sont sauvegardées l'une après l'autre, dans la même exécution et dans le même dossier daté	Pareil
Combien on en garde	Les 7 dernières (les plus anciennes sont supprimées automatiquement)	7 par jour, 4 par semaine, 12 par mois
Où	Le dossier `backups/` du projet, exclu de Git	Règle 3-2-1 : une copie sur le serveur, une sur un autre support, une ailleurs
Récupération	`npm run restore` (`pg_restore --clean` et `mongorestore --drop`)	Pareil, avec un test chaque mois sur une base de test
Objectifs	Au pire, on perd 24 h de données (RPO), et on remet tout en route en quelques minutes (RTO), car la restauration tient en une commande	Un RPO à la minute près, avec l'archivage continu des journaux de PostgreSQL (WAL)

Ce que j'ai vérifié

Test	Résultat
Sauvegarde	Le dossier `AAAA-MM-JJ_HH-MM/` est créé avec `postgres.dump` (environ 19 Ko) et `mongo.archive.gz` (environ 2 Ko) pour les données de démo
Restauration après avoir tout abîmé exprès (un jeu supprimé, un titre modifié, un jeu ajouté, 18 configurations MongoDB supprimées)	On retrouve exactement l'état de départ : 30 jeux, 78 entrées de librairie, 55 configurations. Le jeu ajouté après la sauvegarde n'est plus là
Cohérence après la restauration	Les 55 `config_id` mènent au bon document MongoDB, et toutes les routes de l'API marchent
Rotation	Avec 9 sauvegardes, les 2 plus anciennes sont supprimées et il en reste 7
Tâche planifiée	Lancée à la main : code de retour 0 et nouvelle sauvegarde créée. La prochaine est le lendemain à 3 h

La limite du POC : la tâche ne tourne que si le PC est allumé, branché sur secteur (Windows ne lance pas les tâches planifiées sur batterie par défaut), la session ouverte et Docker lancé. La nuit du 28/10, la session n'était pas ouverte : la tâche n'a pas pu se lancer, ce que `schtasks /Query` montre bien. Sur un serveur allumé en permanence, avec `cron`, ce problème n'existe pas.

8.2 Sécurité et continuité

- **La librairie est la donnée la plus importante** : perdre « quel joueur possède quel jeu », c'est faire perdre leurs jeux aux joueurs. Dans une version commerciale, ce seraient des achats, avec des conséquences financières et juridiques. Ces données sont dans PostgreSQL et sauvegardées tous les jours.
- **Les comptes** (emails, hash des mots de passe) sont des données personnelles. Les sauvegardes sont donc exclues de Git (`.gitignore`) et ne doivent jamais être partagées.
- **Les configurations** sont moins importantes (un joueur peut refaire ses réglages), mais on les sauvegarde au même moment pour rester cohérent avec les `config_id`.
- PostgreSQL et MongoDB sont sauvegardés **dans la même exécution** et restaurés ensemble, donc une restauration redonne deux bases cohérentes entre elles (vérifié : aucun lien cassé).
- Une sauvegarde qu'on n'a jamais testée ne sert à rien : j'ai testé la restauration sur des données abîmées exprès.

---

9. Conception de l'API

9.1 Ce que demande l'énoncé

Besoin de l'énoncé	Route	Qui peut l'utiliser
Créer un compte	`POST /api/auth/register`	Tout le monde
Se connecter	`POST /api/auth/login`	Tout le monde
Avoir les infos d'un jeu	`GET /api/games/:id` (et `GET /api/games`)	Tout le monde
Ajouter un jeu à l'application	`POST /api/games`	Admin
Modifier un jeu	`PATCH /api/games/:id`	Admin
Supprimer un jeu	`DELETE /api/games/:id`	Admin
Ajouter un jeu à la librairie d'un utilisateur	`POST /api/users/:userId/library`	Admin
Voir sa librairie ou en retirer un jeu	`GET` et `DELETE /api/users/:userId/library…`	Le joueur lui-même ou un admin
Voir ou modifier la configuration d'un jeu	`GET` et `PUT /api/users/:userId/library/:gameId/config`	Le joueur lui-même ou un admin

Les URL décrivent des ressources (« la librairie de l'utilisateur 2 »), le verbe HTTP décrit l'action, et les codes de réponse suivent la norme (200, 201, 204, 400, 401, 403, 404, 409).

9.2 Le choix des technos

- **Express** : c'est le framework Node.js le plus utilisé, il est léger, et il convient à un POC qu'il faut livrer vite et que toute l'équipe doit comprendre. Express 5 gère en plus les erreurs des fonctions `async`.
- **PostgreSQL et Sequelize** : pour les données liées entre elles et leur intégrité. C'est l'ORM vu en cours, avec des migrations versionnées.
- **MongoDB et Mongoose** : pour des documents de forme libre, les configurations.
- **JWT** : une authentification sans session, pratique pour une API utilisée par différents clients.

---

10. Sécurité de l'API

10.1 Authentification et autorisation

- **Connexion** : on vérifie l'email et le mot de passe (`bcrypt.compare` avec le hash). Le message d'erreur est le même que l'email n'existe pas ou que le mot de passe soit faux, pour ne pas révéler quels comptes existent.
- **Token JWT** : il est signé avec une clé secrète (`JWT_SECRET`, 96 caractères aléatoires, dans le `.env`), il contient l'id et les rôles, et il est valable 2 heures.

- **Stateless** : le serveur ne garde aucune session. À chaque requête, le client prouve qui il est avec le token (`Authorization: Bearer <token>`).
- **Autorisation** avec des middlewares : `authRequired` (token valide, sinon 401), `requireAdmin` (sinon 403) et `requireOwnerOrAdmin` (on n'accède qu'à sa propre librairie, sinon 403).
- Ce que j'ai vérifié : un token modifié (rôle admin ajouté à la main), expiré ou inventé est refusé. Un joueur ne peut pas lire la librairie d'un autre, ni créer un jeu, ni s'ajouter un jeu lui-même.

10.2 Outils et bonnes pratiques

- Des bibliothèques standard : `jsonwebtoken` (JWT, RFC 7519) et `bcryptjs`.
- Toutes les entrées sont vérifiées, les champs inconnus ignorés et les id dans les URL contrôlés.
- Les clés qui commencent par `$` sont refusées au premier niveau des réglages (injection NoSQL). Le risque reste faible de toute façon : les recherches MongoDB n'utilisent que `userId` et `gameId`, des entiers déjà vérifiés, jamais les réglages.
- Le détail des erreurs internes n'est jamais renvoyé au client, et l'en-tête `X-Powered-By: Express` est désactivé pour ne pas annoncer la technologie utilisée.
- Tout passe en HTTPS (voir 4.2).
- Les secrets ne sont pas dans le code (`.env` et `certs/`, exclus de Git).

> **Pas encore fait** : CORS (origines autorisées), en-têtes de sécurité HTTP (helmet) et limite des tentatives de connexion (rate limiting).

---

11. Tests

Test de charge : fait avec JMeter, voir la partie 5.3 (le scénario, les paliers de 10 à 100 joueurs, les résultats et ce que j'en retiens).

---

12. Documentation

- **Swagger** : https://localhost:3000/api/docs. Chaque route y est décrite (paramètres, corps attendu, réponses possibles, exemples), et on peut la tester directement, même les routes protégées grâce au bouton *Authorize*. La description est au format standard OpenAPI 3 (`docs/openapi.json`).
- **Le README** : l'installation, le lancement de l'API, les comptes de test et toutes les commandes.
- **Cette annexe** : l'explication des choix techniques.